

```
import os
import sys
import ctypes
import tkinter as tk
from tkinter import font as tkfont
from pylogix import PLC

#=====
# ENGIE OPERAÇÃO E MANUTENÇÃO
# PROGRAMADO POR GEOVANE FERRAZ MONTEIRO, 18/05/2026
# PROIBIDA A DISTRIBUIÇÃO SEM AVISO PRÉVIO E CONCENTIMENTO
# =====
# FUNÇÃO PARA CARREGAR FONTE EM TEMPO DE EXECUÇÃO (SEM INSTALAR NO WINDOWS)
# =====
def carregar_fonte_local(caminho_fonte):
    """Carrega temporariamente um arquivo .ttf na memória do Windows para o script usar."""
    if sys.platform == "win32":
        if os.path.exists(caminho_fonte):
            FR_PRIVATE = 0x10
            retorno = ctypes.windll.gdi32.AddFontResourceExW(caminho_fonte, FR_PRIVATE, 0)
            if retorno != 0:
                print(f"Font carregada com sucesso a partir de: {caminho_fonte}")
                return True
            else:
                print("Falha ao registrar a fonte via API do Windows.")
        else:
            print(f"Aviso: Arquivo de fonte '{caminho_fonte}' não foi encontrado na pasta.")
    else:
        print("Carregamento dinâmico automático configurado apenas para Windows.")
    return False

# =====
# CONFIGURAÇÕES DO CLP E ACESSO
# =====
IP_CLP = '172.16.35.30'
SLÖT_CLP = 2

TAGS_PARA_LER = [
    'relogio[3]', # Hora
    'relogio[4]', # Minuto
    'relogio[5]', # Segundo
    'relogio[2]', # Dia
    'relogio[1]', # Mês
    'relogio[0]', # Ano
    'Conta[0].acc',
    'Conta[1].acc',
    'Reais[0]',
    'Reais[1]',
    'Reais[2]',
    'Reais[3]',
    'Reais[4]',
    'Reais[5]'
]

INTERVALO_ATUALIZACAO = 100 # Tempo em milissegundos (100ms = 10Hz)

ARQUIVO_FONTE = "LCDAT&TPhoneTimeDate.ttf"
NOME_INTERNO_FONTE = "LCD AT&T Phone Time/Date"

carregar_fonte_local(ARQUIVO_FONTE)

# =====
# CLASSE PRINCIPAL DA CLASSE VISUAL (GUI)
```

```
# =====
class MonitorCLPApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Supervisório CLP - pylogix")
        self.root.geometry("750x890")
        self.root.configure(bg="#1e1e1e")

        # -----
        # SELEÇÃO DE FONTES CUSTOMIZADAS
        # -----
        self.fonte_titulo = tkfont.Font(family="Consolas", size=16, weight="bold")
        self.fonte_cabecalho = tkfont.Font(family="Consolas", size=11, weight="bold")
        self.fonte_dados = tkfont.Font(family="Consolas", size=11, weight="normal")
        self.fonte_lcd_grande = tkfont.Font(family=NOME_INTERNO_FONTE, size=44,
weight="bold")
        self.fonte_lcd_media = tkfont.Font(family=NOME_INTERNO_FONTE, size=18, weight="bold")

        # Inicializa o driver do pylogix
        self.comm = PLC()
        self.comm.IPAddress = IP_CLP
        self.comm.ProcessorSlot = SLOT_CLP

        # Variáveis de controle para o Indicador (Heartbeat) e Histórico
        self.heartbeat_state = False
        self.historico_conta0 = [] # Armazena os últimos pontos da tag Conta[0].acc
        self.historico_segundos = [] # Armazena os últimos pontos da tag relógio[5]
(Segundos)
        self.max_pontos_grafico = 50 # Quantidade máxima de pontos na tela

        self.widgets_dados = {}
        self.criar_layout()
        self.root.bind("<Escape>", self.fechar_aplicativo)
        self.atualizar_dados()

    def criar_layout(self):
        """Monta a estrutura visual da tabela, relógio, indicador e gráfico."""
        topo_frame = tk.Frame(self.root, bg="#1e1e1e")
        topo_frame.pack(padx=20, pady=10, fill=tk.X)

        # --- LINHA 1: TÍTULO PRINCIPAL ---
        linha1_frame = tk.Frame(topo_frame, bg="#1e1e1e")
        linha1_frame.pack(fill=tk.X, side=tk.TOP)

        lbl_titulo = tk.Label(
            linha1_frame,
            text="MONITORAMENTO EM TEMPO REAL",
            font=self.fonte_titulo, fg="#00ff66", bg="#1e1e1e"
        )
        lbl_titulo.pack(side=tk.LEFT, pady=(0, 5))

        # --- LINHA 2: IP DO CLP E STATUS DA COMUNICAÇÃO ---
        linha2_frame = tk.Frame(topo_frame, bg="#1e1e1e")
        linha2_frame.pack(fill=tk.X, side=tk.TOP, pady=(5, 0))

        self.lbl_ip = tk.Label(
            linha2_frame,
            text=f"CLP IP: {IP_CLP}",
            font=self.fonte_cabecalho, fg="#9cdcfe", bg="#1e1e1e"
        )
        self.lbl_ip.pack(side=tk.LEFT)
```

```

        self.canvas_led = tk.Canvas(linha2_frame, width=25, height=25, bg="#1e1e1e", bd=0,
highlightthickness=0)
        self.canvas_led.pack(side=tk.RIGHT, padx=(5, 0))
        self.led_id = self.canvas_led.create_oval(3, 3, 22, 22, fill="#444444",
outline="#666666")

        lbl_led_texto = tk.Label(linha2_frame, text="Status da comunicação:",
font=self.fonte_cabecalho, fg="ffffff", bg="#1e1e1e")
        lbl_led_texto.pack(side=tk.RIGHT)

# -----
# PAINEL DO RELÓGIO DIGITAL UNIFICADO
# -----
        frame_relogio = tk.LabelFrame(self.root, text=" HORÁRIO DO CLP ",
font=self.fonte_cabecalho, fg="ffffff", bg="#151515", bd=2, relief=tk.GROOVE)
        frame_relogio.pack(padx=20, pady=5, fill=tk.X)

        self.lbl_relogio_hora = tk.Label(frame_relogio, text="--:--:--",
font=self.fonte_lcd_grande, fg="#00ffff", bg="#151515")
        self.lbl_relogio_hora.pack(pady=2)

        self.lbl_relogio_data = tk.Label(frame_relogio, text="DATA: --/--/----",
font=self.fonte_dados, fg="#888888", bg="#151515")
        self.lbl_relogio_data.pack(pady=2)

# -----
# PAINEL DAS VARIÁVEIS (TABELA DINÂMICA)
# -----
        frame_tabela = tk.LabelFrame(self.root, text=" VARIÁVEIS DE PROCESSO ",
font=self.fonte_cabecalho, fg="ffffff", bg="#1e1e1e", bd=0)
        frame_tabela.pack(padx=20, pady=5, fill=tk.X)

        frame_tabela.columnconfigure(0, weight=2)
        frame_tabela.columnconfigure(1, weight=1)
        frame_tabela.columnconfigure(2, weight=1)

        cabecalhos = ["TAG", "Valor", "Status"]
        for col_idx, texto in enumerate(cabecalhos):
            lbl = tk.Label(
                frame_tabela, text=texto, font=self.fonte_cabecalho,
                anchor="center", fg="ffffff", bg="#2d2d2d", relief=tk.FLAT
            )
            lbl.grid(row=0, column=col_idx, sticky="nsew", padx=1, pady=2, ipady=4)

        tags_exibicao = ['Conta[0].acc', 'Conta[1].acc'] + [f'Reais[{i}]' for i in range(6)]

        for row_idx, tag in enumerate(tags_exibicao, start=1):
            cor_fundo = "#252526" if row_idx % 2 == 0 else "#2d2d2d"

            lbl_name = tk.Label(frame_tabela, text=tag, font=self.fonte_dados, fg="#9cdcfe",
bg=cor_fundo, anchor="w")
            lbl_name.grid(row=row_idx, column=0, sticky="nsew", padx=1, pady=1, ipadx=5,
ipady=4)

            cor_valores = "#00ffff" if "Reais" in tag else "#ff9900"
            lbl_value = tk.Label(frame_tabela, text="---", font=self.fonte_lcd_media,
fg=cor_valores, bg=cor_fundo, anchor="center")
            lbl_value.grid(row=row_idx, column=1, sticky="nsew", padx=1, pady=1)

            status_container = tk.Frame(frame_tabela, bg=cor_fundo)
            status_container.grid(row=row_idx, column=2, sticky="nsew", padx=1, pady=1)

```

```

        canvas_tag_led = tk.Canvas(status_container, width=20, height=20, bg=cor_fundo,
bd=0, highlightthickness=0)
        canvas_tag_led.pack(expand=True)

        tag_led_id = canvas_tag_led.create_oval(3, 3, 17, 17, fill="#444444",
outline="#555555")

        self.widgets_dados[tag.lower()] = {
            "value_widget": lbl_value,
            "canvas_widget": canvas_tag_led,
            "led_id": tag_led_id
        }

# --- BOTÃO PARA ZERAR CONTADORES ---
btn_frame = tk.Frame(self.root, bg="#1e1e1e")
btn_frame.pack(padx=20, pady=5, fill=tk.X)

self.btn_zerar = tk.Button(
    btn_frame,
    text="ZERAR CONTADORES (CONTA[0] E CONTA[1])",
    font=self.fonte_cabecalho,
    fg="#1e1e1e", bg="#ff9900",
    activebackground="#cc7a00", activeforeground="#1e1e1e",
    bd=0, height=1, cursor="hand2",
    command=self.zerar_contadores
)
self.btn_zerar.pack(fill=tk.X, ipady=4)

# -----
# PAINEL DO HISTÓRICO GRÁFICO (Canvas Real-Time)
# -----
frame_grafico = tk.LabelFrame(self.root, text=" TENDÊNCIA EM TEMPO REAL ( Laranja
Pontilhado: Conta[0].acc | Ciano: Segundos ) ", font=self.fonte_cabecalho, fg="#ffffff",
bg="#1e1e1e", bd=1, relief=tk.SOLID)
frame_grafico.pack(padx=20, pady=10, fill=tk.BOTH, expand=True)

self.canvas_grafico = tk.Canvas(frame_grafico, bg="#111111", bd=0,
highlightthickness=0)
self.canvas_grafico.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

def zerar_contadores(self):
    """Escreve o valor zero nas variáveis Conta[0].acc e Conta[1].acc via pylogix."""
    try:
        self.btn_zerar.config(state=tk.DISABLED, bg="#555555")
        self.root.update_idletasks()

        res0 = self.comm.Write('Conta[0].acc', 0)
        res1 = self.comm.Write('Conta[1].acc', 0)

        if res0 and res1 and res0.Status == "Success" and res1.Status == "Success":
            self.lbl_ip.config(text=f"CLP IP: {IP_CLP} - CONTADORES ZERADOS OK!",
fg="#00ff66")
            self.historico_conta0.clear()
        else:
            self.lbl_ip.config(text=f"CLP IP: {IP_CLP} - FALHA AO ZERAR (PROIBIDO PELO
CLP)", fg="#ff0000")
        except Exception:
            self.lbl_ip.config(text=f"CLP IP: {IP_CLP} - ERRO DE ESCRITA", fg="#ff0000")
        finally:
            self.root.after(1500, lambda: [
                self.btn_zerar.config(state=tk.NORMAL, bg="#ff9900"),
                self.lbl_ip.config(text=f"CLP IP: {IP_CLP}", fg="#9cdcfe")
            ])

```

```
    ]))

def atualizar_dados(self):
    """Efetua a leitura física no CLP via pylogix de forma segura e resiliente."""
    retorno = None
    sucesso_comunicacao = False
    valores_lidos = {}

    try:
        retorno = self.comm.Read(TAGS_PARA_LER)

        # Validação se o retorno é uma lista válida de objetos de resposta do pylogix
        if retorno and isinstance(retorno, list) and hasattr(retorno[0], 'Status'):
            sucesso_comunicacao = True
            for resposta in retorno:
                if hasattr(resposta, 'Status') and resposta.Status == "Success":
                    valores_lidos[resposta.TagName.lower()] = resposta.Value
                else:
                    sucesso_comunicacao = False
            else:
                sucesso_comunicacao = False
    except Exception:
        sucesso_comunicacao = False

    # --- PROCESSAMENTO DO STATUS DA COMUNICAÇÃO ---
    if sucesso_comunicacao:
        cor_led = "#00ff00" if self.heartbeat_state else "#00aa00"
        self.heartbeat_state = not self.heartbeat_state
        self.canvas_led.itemconfig(self.led_id, fill=cor_led)

        # Atualização do Bloco de Horário
        try:
            hora = valores_lidos.get('relogio[3]', 0)
            minuto = valores_lidos.get('relogio[4]', 0)
            segundo = valores_lidos.get('relogio[5]', 0)
            dia = valores_lidos.get('relogio[2]', 1)
            mes = valores_lidos.get('relogio[1]', 1)
            ano = valores_lidos.get('relogio[0]', 2026)

            self.lbl_relogio_hora.config(text=f"{hora:02d}:{minuto:02d}:{segundo:02d}")
            self.lbl_relogio_data.config(text=f"DATA: {dia:02d}/{mes:02d}/{ano:04d}")

            self.historico_segundos.append(segundo)
            if len(self.historico_segundos) > self.max_pontos_grafico:
                self.historico_segundos.pop(0)
        except Exception:
            pass

        # Atualização individual das Tags válidas
        for resposta in retorno:
            if hasattr(resposta, 'TagName') and resposta.TagName.lower() in self.widgets_dados:
                tag_nome_low = resposta.TagName.lower()
                widgets = self.widgets_dados[tag_nome_low]

                if resposta.Status == "Success":
                    if isinstance(resposta.Value, float):
                        texto_valor = f"{resposta.Value:.2f}"
                    else:
                        texto_valor = str(resposta.Value)

                    widgets["value_widget"].config(text=texto_valor)
```

```
        widgets["canvas_widget"].itemconfig(widgets["led_id"],
fill="#00ff66") # Verde Claro

        if tag_nome_low == "conta[0].acc":
            self.historico_conta0.append(resposta.Value)
            if len(self.historico_conta0) > self.max_pontos_grafico:
                self.historico_conta0.pop(0)
        else:
            widgets["value_widget"].config(text="---")
            widgets["canvas_widget"].itemconfig(widgets["led_id"],
fill="#004d1a") # Verde Escuro

    else:
        # --- TRATAMENTO SEGURO DE FALHA DE LEITURA (OFFLINE/CONEXÃO PERDIDA) ---
        self.canvas_led.itemconfig(self.led_id, fill="#ff0000") # LED Principal Vermelho
        self.lbl_relogio_hora.config(text="--:--:--")
        self.lbl_relogio_data.config(text="DATA: --/--/----")

        # Aplica o reset visual de erro (Verde Escuro) em todos os registros da tabela
imediatamente
        for tag_low, widgets in self.widgets_dados.items():
            widgets["value_widget"].config(text="---")
            widgets["canvas_widget"].itemconfig(widgets["led_id"], fill="#004d1a") #
Verde Escuro (Fail)

        # Atualiza a tela gráfica mesmo sem comunicação (mostra o histórico congelado ou
estático)
        self.desenhar_grafico()

        # Mantém o loop ativo infinitamente, permitindo a autoreconexão quando o cabo for
plugado
        self.root.after(INTERVALO_ATUALIZACAO, self.atualizar_dados)

def desenhar_grafico(self):
    """Desenha as linhas de tendência do histórico coletado dentro do Canvas."""
    self.canvas_grafico.delete("all")

    largura_total = self.canvas_grafico.winfo_width()
    altura = self.canvas_grafico.winfo_height()

    if largura_total < 50 or altura < 10:
        return

    margem_esquerda = 45
    largura_grafico = largura_total - margem_esquerda

    valores_escala = [0, 25, 50, 75, 100]
    for valor in valores_escala:
        y_pos = 15 + ((100 - valor) / 100) * (altura - 30)

        self.canvas_grafico.create_text(
            margem_esquerda - 10, y_pos,
            text=str(valor), font=self.fonte_cabecalho,
            fill="#aaaaaa", anchor="e"
        )

        self.canvas_grafico.create_line(
            margem_esquerda, y_pos, largura_total, y_pos,
            fill="#aaaaaa", dash=(2, 4)
        )

    def plotar_pena(dados, cor, max_escala, pontilhada=False):
```

```
        if len(dados) < 2:
            return

        pontos = []
        fator_x = largura_grafico / (self.max_pontos_grafico - 1)

        for idx, valor in enumerate(dados):
            x = margem_esquerda + (idx * fator_x)
            norm_val = max(0, min(valor, max_escala))
            y = 15 + ((max_escala - norm_val) / max_escala) * (altura - 30)
            pontos.append((x, y))

        estilo_linha = (4, 4) if pontilhada else ()

        for i in range(len(pontos) - 1):
            self.canvas_grafico.create_line(
                pontos[i][0], pontos[i][1],
                pontos[i+1][0], pontos[i+1][1],
                fill=cor, width=2, dash=estilo_linha
            )

        plotar_pena(self.historico_conta0, "#ff9900", max_escala=60, pontilhada=True)
        plotar_pena(self.historico_segundos, "#00ffff", max_escala=60, pontilhada=False)

    def fechar_aplicativo(self, event=None):
        """Encerra a conexão de rede e fecha o loop de eventos."""
        self.comm.Close()
        self.root.destroy()

# =====
# INICIALIZAÇÃO DO PROGRAMA
# =====
if __name__ == "__main__":
    janela_raiz = tk.Tk()
    app = MonitorCLPApp(janela_raiz)
    janela_raiz.mainloop()
```